

HackerRank Preparation Pack

C# Software Engineer, Novel Trading Opportunities

Original single-question practice problems and timed assessment tactics tailored to a market-making, new-products, pricing, connectivity, and trading-systems profile.

Target language	C# / .NET, with performance-aware implementation discipline
Likely assessment style	One substantial coding problem in a 120-minute online assessment
Emphasis areas	Correctness, simple design under complex rules, event processing, ordered state, clean C# code, edge-case discipline
Important note	This pack does not use leaked, private, or recently used HackerRank questions. All practice problems are original.

Use this as a print-ready warm-up packet. The real assessment may differ, but these exercises mirror the likely signal: implementing a dependable, efficient solution to a complex trading-systems style problem under time pressure.

Section 1 — Role Skill Map

The NTO role points to engineers who can move quickly in ambiguous trading domains while still writing clean, correct, scalable software. The most likely HackerRank signal is a single, compound problem that tests modelling, algorithm choice, careful state management, and implementation quality in C#.

Area	Signals from the role and profile context
Algorithms and data structures	Arrays, sorting, hash maps, queues/deques, heaps, ordered dictionaries, prefix sums, Fenwick trees, graph traversal/topological ordering, greedy selection, sliding windows, and state-machine modelling.
Coding and debugging skills	Translate dense rules into invariants, write deterministic code quickly, avoid off-by-one errors, handle repeated values and ties, validate with small adversarial tests, and keep code readable under time pressure.
System or domain knowledge	Event-driven trading workflows, venue/exchange connectivity, order lifecycle consistency, pricing dependencies, risk limits, time-to-market trade-offs, reliable real-time data processing, and low-latency pragmatism.
Language-specific skills	C# collections, custom comparers, PriorityQueue or SortedDictionary usage, value types, long vs int discipline, StringBuilder output, fast input parsing, low-allocation loops, and awareness of GC pressure in hot paths.
Profile-informed emphasis	The supplied profile suggests strong alignment with .NET trading systems, low-latency execution, smart order routing, GC tuning, event-driven architecture, and real-time analytics. Preparation should therefore lean into compound stateful problems rather

	than pure academic puzzles.
--	-----------------------------

Section 2 — Assessment Blueprint

Because the reported format is two hours for one solution, expect one substantial problem rather than several short puzzles. The problem may still contain multiple sub-behaviours: parsing events, maintaining state, answering queries, and proving the selected data structure is efficient enough.

- Event stream with state transitions: a natural fit for trading systems where order, venue, quote, or risk events arrive over time.
- Ordered-state optimisation: route to best venues, select best quotes, maintain best prices, or compute lowest-cost fills using heaps, sorted dictionaries, or indexed trees.
- Pricing or dependency propagation: new products and pricing infrastructure map well to DAG/topological problems and incremental recalculation.
- Risk-aware validation: kill switches, notional limits, stale data, lifecycle reconciliation, and invariant checking are plausible because the role emphasizes principled, risk-conscious delivery.
- C# implementation quality: fast scanner, StringBuilder, careful long arithmetic, and avoiding accidental high-allocation designs can matter as much as the headline algorithm.

Predicted single-question structure

Phase	Expected challenge	What it tests
First 20 minutes	Understand the model and constraints	Can you identify whether the core is sorting, graph, sliding window, heap, or indexed aggregation?
Middle 70 minutes	Implement the main algorithm and state updates	Can you keep the code simple, correct, and performant in C#?
Final 30 minutes	Adversarial tests and cleanup	Can you catch overflow, tie-breaking, stale state, repeated events, and boundary windows?

Difficulty mix for the practice set

Question	Expected difficulty	Why it is relevant
Q1	Medium	Greedy routing and exact arithmetic
Q2	Medium	Rolling risk limits and event windows
Q3	Medium	Lifecycle state machine and invariant checking
Q4	Medium-hard	Dependency graph and deterministic recomputation
Q5	Hard	Venue-style quote book with expiries, cancels, and indexed best-fill queries

Section 3 — Timed Practice Set (Original Questions)

Each of the five questions below is original and designed as a 120-minute single-problem rehearsal. In practice, attempt one question per timed session, then review the reference solution and edge cases.

Question 1 — Venue Fill Planner

Difficulty: Medium | **Recommended timing:** 35-45 minutes

You need to route a parent order across available venues. For a BUY order, each venue has unit cost price + fee. For a SELL order, each venue has unit value price - fee. Fill the target quantity using the best venues first. Ties must be broken by lower latency, then lower venue id. Output the total cost for BUY or total proceeds for SELL, followed by the venue quantities used. If the target cannot be fully filled, print IMPOSSIBLE.

Input format

- The first line contains side, targetQty, and N. side is BUY or SELL.
- Each of the next N lines contains venueId, price, fee, availableQty, latency.
- price and fee are integer ticks, not decimals.

Output format

- If a full fill is impossible, print IMPOSSIBLE.
- Otherwise print total value, then the number of venues used, then one line per venue: venueId quantity.

Constraints

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq \text{targetQty}, \text{availableQty} \leq 10^9$
- $0 \leq \text{fee} \leq \text{price} \leq 10^9$
- Use 64-bit arithmetic for totals.

Sample cases

Sample 1

```
Input
BUY 100 4
1 101 1 40 20
2 100 3 70 10
3 99 4 80 5
4 102 0 10 1

Output
10220
3
1 40
2 60
```

Sample 2

```
Input
SELL 50 2
9 200 1 20 3
8 198 0 10 1

Output
IMPOSSIBLE
```

Edge cases to watch

- For BUY, lower unit value is better; for SELL, higher unit value is better.
- A venue may have zero contribution if the target is already filled.
- Tie-break output order matters because the selected venues must follow the sorted priority.
- Totals can exceed 32-bit integer range.

Approach

Sort venues by the exact routing priority and greedily consume capacity until the target is met. This is optimal because each unit from a better venue is always at least as good as each unit from any later venue, and venue capacities are independent.

Reference solution (C#)

```
using System;
using System.Collections.Generic;

class Program
{
    struct Venue
    {
        public int Id;
        public long UnitValue;
        public long Quantity;
        public long Latency;
    }

    static void Main()
    {
        var fs = new FastScanner();
        string side = fs.Next();
        long target = fs.NextLong();
        int n = fs.NextInt();
        var venues = new List<Venue>();

        for (int i = 0; i < n; i++)
        {
            int id = fs.NextInt();
            long price = fs.NextLong();
            long fee = fs.NextLong();
            long qty = fs.NextLong();
            long latency = fs.NextLong();
            long unit = side == "BUY" ? price + fee : price - fee;
            venues.Add(new Venue { Id = id, UnitValue = unit, Quantity = qty, Latency = latency });
        }

        venues.Sort((a, b) =>
        {
            int cmp = side == "BUY" ? a.UnitValue.CompareTo(b.UnitValue) : b.UnitValue.CompareTo(a.UnitValue);
            if (cmp != 0) return cmp;
            cmp = a.Latency.CompareTo(b.Latency);
            if (cmp != 0) return cmp;
            return a.Id.CompareTo(b.Id);
        });

        long remaining = target;
        long total = 0;
        var used = new List<(int id, long qty)>();

        foreach (var v in venues)
        {
            if (remaining == 0) break;
            long take = Math.Min(remaining, v.Quantity);
            if (take == 0) continue;
            total += take * v.UnitValue;
            remaining -= take;
            used.Add((v.Id, take));
        }

        if (remaining > 0)
        {
            Console.WriteLine("IMPOSSIBLE");
            return;
        }
    }
}
```

```

        Console.WriteLine(total);
        Console.WriteLine(used.Count);
        foreach (var x in used) Console.WriteLine($"{x.id} {x.qty}");
    }

    class FastScanner
    {
        private readonly byte[] data = new byte[1 << 16];
        private int len, ptr;
        private int Read() { if (ptr >= len) { len = Console.OpenStandardInput().Read(data, 0, data.Length); ptr = 0; if (len == 0) return -1; } return data[ptr++]; }
        public string Next() { int c; do { c = Read(); } while (c <= 32 && c >= 0); var chars = new List<char>(); while (c > 32) { chars.Add((char)c); c = Read(); } return new string(chars.ToArray()); }
        public int NextInt() => (int)NextLong();
        public long NextLong() { int c; do { c = Read(); } while (c <= 32 && c >= 0); int sign = 1; if (c == '-') { sign = -1; c = Read(); } long v = 0; while (c > 32) { v = v * 10 + c - '0'; c = Read(); } return v * sign; }
    }
}

```

Complexity

Time: $O(N \log N)$. Space: $O(N)$.

Question 2 — Rolling Risk Kill Switch

Difficulty: Medium | **Recommended timing:** 35-45 minutes

A strategy receives fills in non-decreasing timestamp order. A kill switch fires at the earliest fill timestamp t where the rolling window $[t - W, t]$ has either absolute net quantity greater than Q or total traded notional greater than L . Buy fills add quantity; sell fills subtract quantity. Find the earliest trigger time, or -1 if no trigger occurs.

Input format

- The first line contains N , W , Q , and L .
- Each of the next N lines contains timestamp, side, quantity, price. side is B or S.

Output format

- Print the earliest timestamp where either limit is breached; otherwise print -1.

Constraints

- $1 \leq N \leq 2 \cdot 10^5$
- $0 \leq W \leq 10^9$
- $1 \leq \text{quantity, price} \leq 10^9$
- Timestamps are non-decreasing.

Sample cases

Sample 1

Input
5 10 100 10000
1 B 40 50
4 B 50 60
9 S 10 70
12 B 30 80
30 S 200 10
Output
12

Sample 2

Input
4 0 10 1000
5 B 4 10
5 S 3 20
6 B 9 10
7 B 9 10
Output
-1

Edge cases to watch

- $W = 0$ keeps only fills at the exact current timestamp.
- Repeated timestamps are allowed.
- The window is inclusive on both ends.
- Notional is quantity times price regardless of side.

Approach

Use a queue for active fills in the rolling window. Add the new fill, remove fills with timestamp less than $t - W$, update rolling net quantity and rolling notional, then test the two limits. The first time a limit is breached is the answer because input time is non-decreasing.

Reference solution (C#)

```

using System;
using System.Collections.Generic;

class Program
{
    struct Fill
    {
        public long T, SignedQty, Notional;
    }

    static void Main()
    {
        var fs = new FastScanner();
        int n = fs.NextInt();
        long window = fs.NextLong();
        long maxAbsQty = fs.NextLong();
        long maxNotional = fs.NextLong();

        var q = new Queue<Fill>();
        long rollingQty = 0;
        long rollingNotional = 0;

        for (int i = 0; i < n; i++)
        {
            long t = fs.NextLong();
            string side = fs.Next();
            long qty = fs.NextLong();
            long price = fs.NextLong();
            long signed = side == "B" ? qty : -qty;
            long notional = qty * price;

            q.Enqueue(new Fill { T = t, SignedQty = signed, Notional = notional });
            rollingQty += signed;
            rollingNotional += notional;

            while (q.Count > 0 && t - q.Peek().T > window)
            {
                var old = q.Dequeue();
                rollingQty -= old.SignedQty;
                rollingNotional -= old.Notional;
            }

            if (Math.Abs(rollingQty) > maxAbsQty || rollingNotional > maxNotional)
            {
                Console.WriteLine(t);
                return;
            }
        }

        Console.WriteLine(-1);
    }

    class FastScanner
    {
        private readonly byte[] data = new byte[1 << 16];
        private int len, ptr;
        private int Read() { if (ptr >= len) { len = Console.OpenStandardInput().Read(data, 0, data.Length); ptr = 0; if (len == 0) return -1; } return data[ptr++]; }
        public string Next() { int c; do { c = Read(); } while (c <= 32 && c >= 0); var chars = new List<char>(); while (c > 32) { chars.Add((char)c); c = Read(); } return new string(chars.ToArray()); }
        public int NextInt() => (int)NextLong();
        public long NextLong() { int c; do { c = Read(); } while (c <= 32 && c >= 0); int sign = 1; if (c == '-') { sign = -1; c = Read(); } long v = 0; while (c > 32) { v = v * 10 + c - '0'; c = Read(); } return v * sign; }
    }
}

```

```
}
```

Complexity

Time: $O(N)$. Space: $O(N)$ in the worst case for the active window.

Question 3 — Order Lifecycle Reconciler

Difficulty: Medium | **Recommended timing:** 40-50 minutes

An exchange connector emits order lifecycle events. A valid order begins with NEW quantity, then either ACK to become live or REJECT to finish. A live order may receive one or more FILL events with positive quantities not exceeding the remaining quantity; when remaining quantity reaches zero it is done. A live order may also be CANCELLED. Count invalid events and output all orders still pending or live at the end, sorted by order id.

Input format

- The first line contains N.
- Each event is one of: NEW orderId qty, ACK orderId, REJECT orderId, FILL orderId qty, CANCEL orderId.
- orderId is a non-empty token without spaces.

Output format

- Print the number of invalid events.
- Print the number of still-active orders.
- Then print active orders in lexicographic order as orderId remainingQty.

Constraints

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq \text{qty} \leq 10^9$ for NEW and FILL
- Invalid events do not change state.

Sample cases

Sample 1

```
Input
9
NEW 01 100
ACK 01
FILL 01 30
NEW 02 50
REJECT 02
FILL 02 10
FILL 01 70
CANCEL 01
ACK 03
```

```
Output
3
0
```

Sample 2

```
Input
6
NEW A 10
ACK A
FILL A 4
NEW B 7
ACK B
CANCEL B
```

```
Output
0
1
A 6
```

Edge cases to watch

- Duplicate NEW for the same order is invalid, even if the previous order is done.
- A fill that overfills an order is invalid and must not reduce remaining quantity.
- ACK is only valid after NEW and before ACK/REJECT.
- Pending orders count as active at the end.

Approach

Model each order as a state machine. For every event, check whether the transition is allowed from the current state. If not, increment the invalid count and leave the stored state unchanged. At the end, collect pending and live orders and sort their ids.

Reference solution (C#)

```
using System;
using System.Collections.Generic;
using System.Text;

class Program
{
    enum State { None, Pending, Live, Done, Rejected }

    static void Main()
    {
        var fs = new FastScanner();
        int n = fs.NextInt();
        var state = new Dictionary<string, State>();
        var remaining = new Dictionary<string, long>();
        long invalid = 0;

        for (int i = 0; i < n; i++)
        {
            string evt = fs.Next();
            string id = fs.Next();
            long qty = evt == "NEW" || evt == "FILL" ? fs.NextLong() : 0;

            state.TryGetValue(id, out State s);
            bool ok = true;

            if (evt == "NEW")
            {
                if (s != State.None) ok = false;
                else { state[id] = State.Pending; remaining[id] = qty; }
            }
            else if (evt == "ACK")
            {
                if (s != State.Pending) ok = false;
                else state[id] = State.Live;
            }
            else if (evt == "REJECT")
            {
                if (s != State.Pending) ok = false;
                else { state[id] = State.Rejected; remaining[id] = 0; }
            }
            else if (evt == "FILL")
            {
                if (s != State.Live || qty <= 0 || qty > remaining[id]) ok = false;
                else
                {
                    remaining[id] -= qty;
                    if (remaining[id] == 0) state[id] = State.Done;
                }
            }
            else if (evt == "CANCEL")
            {
                if (s != State.Live) ok = false;
            }
        }
    }
}
```

```

        else { state[id] = State.Done; remaining[id] = 0; }
    }

    if (!ok) invalid++;
}

var live = new List<string>();
foreach (var kv in state)
    if (kv.Value == State.Pending || kv.Value == State.Live)
        live.Add(kv.Key);
live.Sort(StringComparer.Ordinal);

var sb = new StringBuilder();
sb.AppendLine(invalid.ToString());
sb.AppendLine(live.Count.ToString());
foreach (var id in live) sb.AppendLine(id + " " + remaining[id]);
Console.WriteLine(sb.ToString());
}

class FastScanner
{
    private readonly byte[] data = new byte[1 << 16];
    private int len, ptr;
    private int Read() { if (ptr >= len) { len = Console.OpenStandardInput().Read(data, 0, data.Length); ptr = 0; if (len == 0) return -1; } return data[ptr++]; }
    public string Next() { int c; do { c = Read(); } while (c <= 32 && c >= 0); var chars = new List<char>(); while (c > 32) { chars.Add((char)c); c = Read(); } return new string(chars.ToArray()); }
    public int NextInt() => (int)NextLong();
    public long NextLong() { int c; do { c = Read(); } while (c <= 32 && c >= 0); int sign = 1; if (c == '-') { sign = -1; c = Read(); } long v = 0; while (c > 32) { v = v * 10 + c - '0'; c = Read(); } return v * sign; }
}

```

Complexity

Time: $O(N + A \log A)$, where A is the number of active orders printed. Space: $O(U)$, where U is the number of distinct order ids.

Question 4 — Pricing Cascade Recalculation

Difficulty: Medium-hard | **Recommended timing:** 50-65 minutes

Pricing components depend on one another. A rule $u \rightarrow v$ means component v uses the output of component u , so u must be recalculated before v . Given a batch of changed components, output the lexicographically smallest valid recalculation order for every impacted component: each changed component and every component reachable from it. If the impacted subgraph contains a cycle, print CYCLE.

Input format

- The first line contains N and M .
- Each of the next M lines contains $u \rightarrow v$, meaning u influences v .
- The next line contains K followed by K changed component ids.

Output format

- If there is a cycle in the impacted subgraph, print CYCLE.
- Otherwise print the number of impacted components, then the recomputation order.

Constraints

- $1 \leq N \leq 2 \cdot 10^5$
- $0 \leq M \leq 2 \cdot 10^5$
- $1 \leq K \leq N$
- $1 \leq u, v \leq N$

Sample cases

Sample 1

Input
6 5
1 3
2 3
3 4
2 5
5 6
1 2
Output
6
1 2 3 4 5 6

Sample 2

Input
4 4
1 2
2 3
3 2
3 4
1 1
Output
CYCLE

Edge cases to watch

- Only the impacted subgraph matters. A cycle outside the impacted set should not affect the answer.
- When multiple components are ready, choose the smallest id.
- Disconnected impacted components are possible.
- A changed component with no dependents still appears in the output.

Approach

First mark all impacted nodes by DFS/BFS from the changed components along dependency edges. Then run Kahn topological sort restricted to impacted nodes. A min-priority queue gives the lexicographically smallest valid recomputation order. If not all impacted nodes are emitted, there is a cycle in the impacted subgraph.

Reference solution (C#)

```
using System;
using System.Collections.Generic;
using System.Text;

class Program
{
    static void Main()
    {
        var fs = new FastScanner();
        int n = fs.NextInt();
        int m = fs.NextInt();
        var g = new List<int>[n + 1];
        var rev = new List<int>[n + 1];
        for (int i = 1; i <= n; i++) { g[i] = new List<int>(); rev[i] = new List<int>(); }

        for (int i = 0; i < m; i++)
        {
            int u = fs.NextInt(), v = fs.NextInt();
            g[u].Add(v);
            rev[v].Add(u);
        }

        int k = fs.NextInt();
        var impacted = new bool[n + 1];
        var stack = new Stack<int>();
        for (int i = 0; i < k; i++)
        {
            int x = fs.NextInt();
            if (!impacted[x]) { impacted[x] = true; stack.Push(x); }
        }

        while (stack.Count > 0)
        {
            int u = stack.Pop();
            foreach (int v in g[u])
                if (!impacted[v]) { impacted[v] = true; stack.Push(v); }
        }

        var indeg = new int[n + 1];
        int need = 0;
        for (int u = 1; u <= n; u++)
        {
            if (!impacted[u]) continue;
            need++;
            foreach (int v in g[u]) if (impacted[v]) indeg[v]++;
        }

        var pq = new PriorityQueue<int, int>();
        for (int i = 1; i <= n; i++) if (impacted[i] && indeg[i] == 0) pq.Enqueue(i, i);

        var order = new List<int>();
        while (pq.Count > 0)
        {
            int u = pq.Dequeue();
            order.Add(u);
            foreach (int v in g[u])
                if (impacted[v] && --indeg[v] == 0)
                    pq.Enqueue(v, v);
        }
    }
}
```

```

    }

    if (order.Count != need)
    {
        Console.WriteLine("CYCLE");
        return;
    }

    Console.WriteLine(order.Count);
    Console.WriteLine(string.Join(" ", order));
}

class FastScanner
{
    private readonly byte[] data = new byte[1 << 16];
    private int len, ptr;
    private int Read() { if (ptr >= len) { len = Console.OpenStandardInput().Read(data, 0, data.Length); ptr = 0; if (len
== 0) return -1; } return data[ptr++]; }
    public int NextInt() => (int)NextLong();
    public long NextLong() { int c; do { c = Read(); } while (c <= 32 && c >= 0); int sign = 1; if (c == '-') { sign = -1;
c = Read(); } long v = 0; while (c > 32) { v = v * 10 + c - '0'; c = Read(); } return v * sign; }
}
}

```

Complexity

Time: $O((N + M) \log N)$ in the worst case. Space: $O(N + M)$.

Question 5 — Expiring Quote Book Queries

Difficulty: Hard | **Recommended timing:** 75-100 minutes

A simplified quote book receives timestamped events in non-decreasing time order. ADD creates a bid or ask quote with id, side, price, quantity, and expiry timestamp. CANCEL removes an active quote. QUERY asks for the best hypothetical fill cost for a BUY or SELL quantity at the current timestamp. BUY queries consume asks from lowest price upward; SELL queries consume bids from highest price downward. Queries do not consume liquidity. Quotes with expiry time strictly less than the current timestamp are inactive. Return -1 if there is not enough active opposite-side quantity.

Input format

- The first line contains N.
- ADD t id side price qty expiry, where side is B for bid or A for ask.
- CANCEL t id.
- QUERY t side qty, where side is BUY or SELL.
- Event timestamps are non-decreasing.

Output format

- For each QUERY, print the minimum BUY cost or maximum SELL proceeds, or -1 if insufficient liquidity.

Constraints

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq \text{price, qty} \leq 10^9$
- $1 \leq \text{id} \leq 10^9$
- Each ADD id is unique.
- Use 64-bit arithmetic.

Sample cases

Sample 1

```
Input
8
ADD 1 10 A 101 50 5
ADD 2 11 A 100 30 10
QUERY 3 BUY 60
CANCEL 4 11
QUERY 4 BUY 60
ADD 6 12 B 99 40 8
ADD 6 13 B 98 50 20
QUERY 7 SELL 70

Output
6030
-1
6910
```

Sample 2

```
Input
5
ADD 1 1 A 10 5 1
QUERY 1 BUY 5
QUERY 2 BUY 1
ADD 3 2 B 8 10 3
QUERY 4 SELL 1

Output
50
-1
```

Edge cases to watch

- Expiry is strict: expiry < current time means inactive; expiry == current time is still active.
- CANCEL for an inactive or unknown quote has no effect.
- Queries are hypothetical and must not reduce book quantity.
- Prices must be compressed before Fenwick tree indexing.

Approach

Read all events once to collect prices for coordinate compression. Maintain active aggregate quantity and aggregate notional at each price using Fenwick trees: asks indexed from low to high and bids indexed from high to low. Also keep quote records by id and a min-priority queue by expiry for lazy expiration. A query first checks total active opposite-side quantity, then uses Fenwick lower_bound to find the marginal price level and compute full-level cost plus partial-level cost.

Reference solution (C#)

```
using System;
using System.Collections.Generic;
using System.Text;

class Program
{
    class Quote
    {
        public int Id, AskIndex, BidIndex;
        public string Side = "";
        public long Price, Qty, Expiry;
        public bool Active;
    }

    class Fenwick
    {
        private readonly long[] bit;
        public Fenwick(int n) { bit = new long[n + 2]; }
        public void Add(int idx, long delta) { for (; idx < bit.Length; idx += idx & -idx) bit[idx] += delta; }
        public long Sum(int idx) { long s = 0; for (; idx > 0; idx -= idx & -idx) s += bit[idx]; return s; }
        public long Total => Sum(bit.Length - 1);
        public int LowerBound(long target)
        {
            int idx = 0;
            int mask = 1;
            while ((mask << 1) < bit.Length) mask <<= 1;
            for (int step = mask; step > 0; step >>= 1)
            {
                int next = idx + step;
                if (next < bit.Length && bit[next] < target)
                {
                    idx = next;
                    target -= bit[next];
                }
            }
            return idx + 1;
        }
    }

    struct Event
    {
        public long T, Price, Qty, Expiry;
        public int Id;
        public string Type, Side;
    }

    static void Main()
    {
```

```

var fs = new FastScanner();
int n = fs.NextInt();
var events = new Event[n];
var prices = new List<long>();

for (int i = 0; i < n; i++)
{
    string type = fs.Next();
    long t = fs.NextLong();
    var e = new Event { Type = type, T = t, Side = "" };
    if (type == "ADD")
    {
        e.Id = fs.NextInt(); e.Side = fs.Next(); e.Price = fs.NextLong(); e.Qty = fs.NextLong(); e.Expiry =
fs.NextLong();
        prices.Add(e.Price);
    }
    else if (type == "CANCEL") e.Id = fs.NextInt();
    else { e.Side = fs.Next(); e.Qty = fs.NextLong(); }
    events[i] = e;
}

prices.Sort();
var uniq = new List<long>();
foreach (var p in prices) if (uniq.Count == 0 || uniq[^1] != p) uniq.Add(p);
int P = uniq.Count;

var asksQty = new Fenwick(P + 2); var asksCost = new Fenwick(P + 2);
var bidsQty = new Fenwick(P + 2); var bidsCost = new Fenwick(P + 2);
var quotes = new Dictionary<int, Quote>();
var expiries = new PriorityQueue<int, long>();
var output = new StringBuilder();

foreach (var e in events)
{
    Expire(e.T, quotes, expiries, asksQty, asksCost, bidsQty, bidsCost);

    if (e.Type == "ADD")
    {
        int asc = Lower(uniq, e.Price) + 1;
        int desc = P - asc + 1;
        var q = new Quote { Id = e.Id, Side = e.Side, Price = e.Price, Qty = e.Qty, Expiry = e.Expiry, Active = true,
AskIndex = asc, BidIndex = desc };
        quotes[e.Id] = q;
        expiries.Enqueue(e.Id, e.Expiry);
        AddQuote(q, q.Qty, asksQty, asksCost, bidsQty, bidsCost);
    }
    else if (e.Type == "CANCEL")
    {
        if (quotes.TryGetValue(e.Id, out var q) && q.Active)
        {
            AddQuote(q, -q.Qty, asksQty, asksCost, bidsQty, bidsCost);
            q.Active = false;
        }
    }
    else
    {
        if (e.Side == "BUY") output.AppendLine(Query(asksQty, asksCost, uniq, e.Qty, false));
        else output.AppendLine(Query(bidsQty, bidsCost, uniq, e.Qty, true));
    }
}
Console.Write(output.ToString());
}

static void Expire(long t, Dictionary<int, Quote> quotes, PriorityQueue<int, long> pq, Fenwick aq, Fenwick ac, Fenwick bq,
Fenwick bc)
{
    while (pq.TryPeek(out int peekId, out long expiry) && expiry < t)

```

```

    {
        int id = pq.Dequeue();
        var q = quotes[id];
        if (q.Active && q.Expiry < t)
        {
            AddQuote(q, -q.Qty, aq, ac, bq, bc);
            q.Active = false;
        }
    }
}

static void AddQuote(Quote q, long deltaQty, Fenwick aq, Fenwick ac, Fenwick bq, Fenwick bc)
{
    if (q.Side == "A") { aq.Add(q.AskIndex, deltaQty); ac.Add(q.AskIndex, deltaQty * q.Price); }
    else { bq.Add(q.BidIndex, deltaQty); bc.Add(q.BidIndex, deltaQty * q.Price); }
}

static string Query(Fenwick qty, Fenwick cost, List<long> ascPrices, long need, bool reversed)
{
    if (qty.Total < need) return "-1";
    int idx = qty.LowerBound(need);
    long qtyBefore = qty.Sum(idx - 1);
    long costBefore = cost.Sum(idx - 1);
    long take = need - qtyBefore;
    int ascIndex = reversed ? ascPrices.Count - idx : idx - 1;
    long price = ascPrices[ascIndex];
    return (costBefore + take * price).ToString();
}

static int Lower(List<long> a, long x)
{
    int l = 0, r = a.Count;
    while (l < r) { int m = (l + r) / 2; if (a[m] < x) l = m + 1; else r = m; }
    return l;
}

class FastScanner
{
    private readonly byte[] data = new byte[1 << 16]; private int len, ptr;
    private int Read() { if (ptr >= len) { len = Console.OpenStandardInput().Read(data, 0, data.Length); ptr = 0; if (len == 0) return -1; } return data[ptr++]; }
    public string Next() { int c; do { c = Read(); } while (c <= 32 && c >= 0); var chars = new List<char>(); while (c > 32) { chars.Add((char)c); c = Read(); } return new string(chars.ToArray()); }
    public int NextInt() => (int)NextLong();
    public long NextLong() { int c; do { c = Read(); } while (c <= 32 && c >= 0); long v = 0; while (c > 32) { v = v * 10 + c - '0'; c = Read(); } return v; }
}

```

Complexity

Time: $O(N \log P)$, where P is the number of distinct prices. Space: $O(N + P)$.

Section 4 — Rapid Strategy Guide

Use this as the operating plan for a 120-minute, one-solution C# assessment.

Triage in the first 10 minutes

- Read the full statement before coding. Identify every object: event, order, quote, venue, dependency, query, and whether events consume state or are hypothetical.
- Convert constraints into a target complexity immediately. N around $2e5$ usually means $O(N \log N)$ or $O(N)$, not nested loops.
- Write down the invariant. Examples: active window contains exactly timestamps in $[t-W, t]$; Fenwick tree totals equal active quote quantity; state transitions never update on invalid events.
- Decide whether the problem is online or offline. If all input can be read first, coordinate compression and pre-sorting may simplify a hard problem.

C# implementation tactics

- Use a fast scanner for large input and `StringBuilder` for large output.
- Use `long` for timestamps, quantities, prices, products, notionals, and cumulative costs.
- Prefer arrays, `List<T>`, `Dictionary<TKey, TValue>`, `Queue<T>`, `Stack<T>`, `SortedDictionary<TKey, TValue>`, and `PriorityQueue<TElement, TPriority>` where they match the invariant directly.
- Be careful with `PriorityQueue` lazy deletion: the heap can contain stale entries, so always re-check current active state after popping.
- For custom ordering, make the tie-breaks explicit and deterministic. Hidden tests often include equal prices, equal timestamps, and repeated quantities.

Testing fast

- Create one minimum-size test, one all-ties test, one expiry/cancel/stale-state test, and one overflow-scale test.
- For windows, test $W = 0$ and repeated timestamps. For graphs, test disconnected components and cycles outside the impacted set. For order books, test partial fill at the marginal price.
- After sample tests pass, scan the code for places where invalid events might accidentally mutate state.

Recommended 120-minute pacing

- 0-10 minutes: model the rules, write target complexity, decide data structures.
- 10-30 minutes: implement parser and core storage structures.
- 30-85 minutes: implement the algorithm, keeping functions small and named by invariant.
- 85-105 minutes: run samples and self-made adversarial tests.
- 105-120 minutes: final pass for overflow, tie-breaking, stale heap entries, and output formatting.

Final reminder

For this role, the strongest assessment signal is not only knowing algorithms. It is the ability to turn a messy, trading-flavoured problem into simple, dependable C# code that handles scale, edge cases, and state correctly.